

กลุ่ม 13 — SLICES & MEMORY MODEL



1. นายปภัทกร มงคลนรกิจ — Concept + Short Code Illustration
2. นางสาวศิริกานต์ หรุ่นมาบแค — Detailed Code + Live Demo
3. นายสิรวิชญ์ เชี่ยวชาญ — Rust vs Other Language + PPL Analysis
4. นายวีรภัทร พุดหอม — Exercises + Common Mistakes + Challenge

Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

Member 1

**Concept + Short Code
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่างสั้น ๆ

~5 นาที

2

Member 2

Detailed Code + Live Demo

อธิบายโค้ดเชิงลึกและสาธิตการทำงานจริง

~5 นาที

3

Member 3

**Rust vs Other Language +
PPL Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง
Programming Language

~5 นาที

4

Member 4

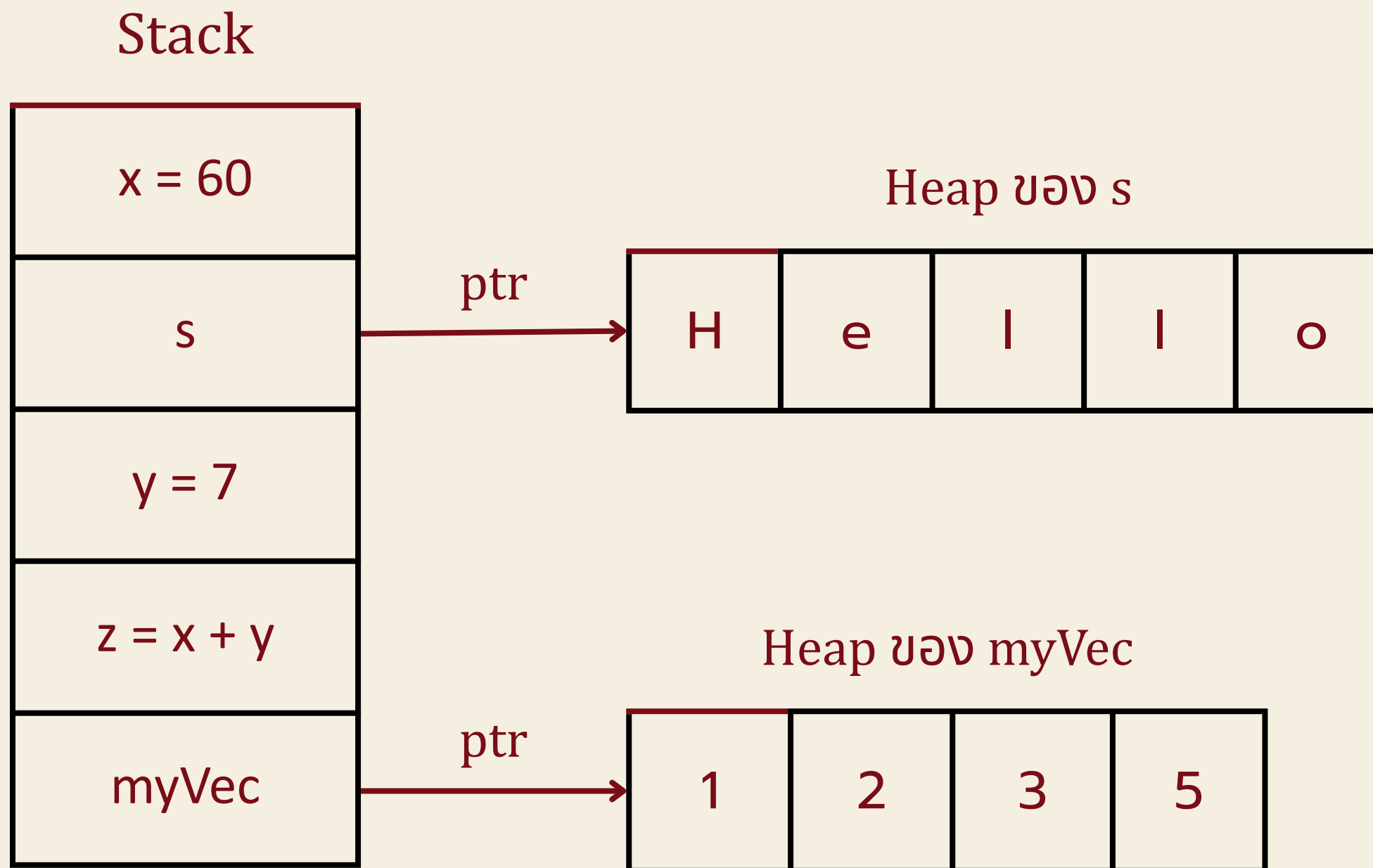
**Exercises + Common
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย และโจทย์ท้าทายผู้ฟัง

~5 นาที

MEMORY MODEL (CONCEPT)

- Memory Model : ชุดการจัดเก็บข้อมูล/ตัวแปร การใช้งาน การคืนความจำ โดยใช้หลัก Ownership , Borrowing และ Lifetime
- ใน Memory Model มีการเก็บข้อมูลอยู่ 2 แบบหลักๆ คือ Stack เก็บข้อมูลรู้ขนาดแน่นอน / Heap เก็บข้อมูลที่ขนาดเปลี่ยนได้



```

1 fn main(){
2     let x = 60;
3     let s = String::from("Hello");
4     let y = 7;
5     let z = x + y;
6     let myVec = vec![1, 2, 3, 5];
7 }

```

ทำไม s และ myVec ถึงอยู่ใน Stack ด้วยล่ะ?

Ans: ต้องเก็บค่าข้อมูลอย่างอื่นแทน เช่น ขนาดข้อมูล , ความจุและตัวชี้ข้อมูลบน Heap (length , capacity , pointer)

MEMORY MODEL (CONCEPT)

- `&str` : การยืมข้อความจากตัวแปรหลัก (`s`) โดยการใช้ `ptr` ชี้ไปยังตัวแปรนั้นๆ (ใช้กับจำพวก `slice`)
- `.clone()` : สร้างก้อน String บน heap ขึ้นมาใหม่ พร้อมคัดลอกข้อความจากตัวแปรที่ต้องการไปยังก้อน heap ใหม่

```

1 fn main() {
2     let s = String::from("Hello");
3     let borrowed: &str = &s;
4     let copied = s.clone();
5     println!("s      = {s}");
6     println!("borrowed= {borrowed}");
7     println!("copied  = {copied}");
8 }

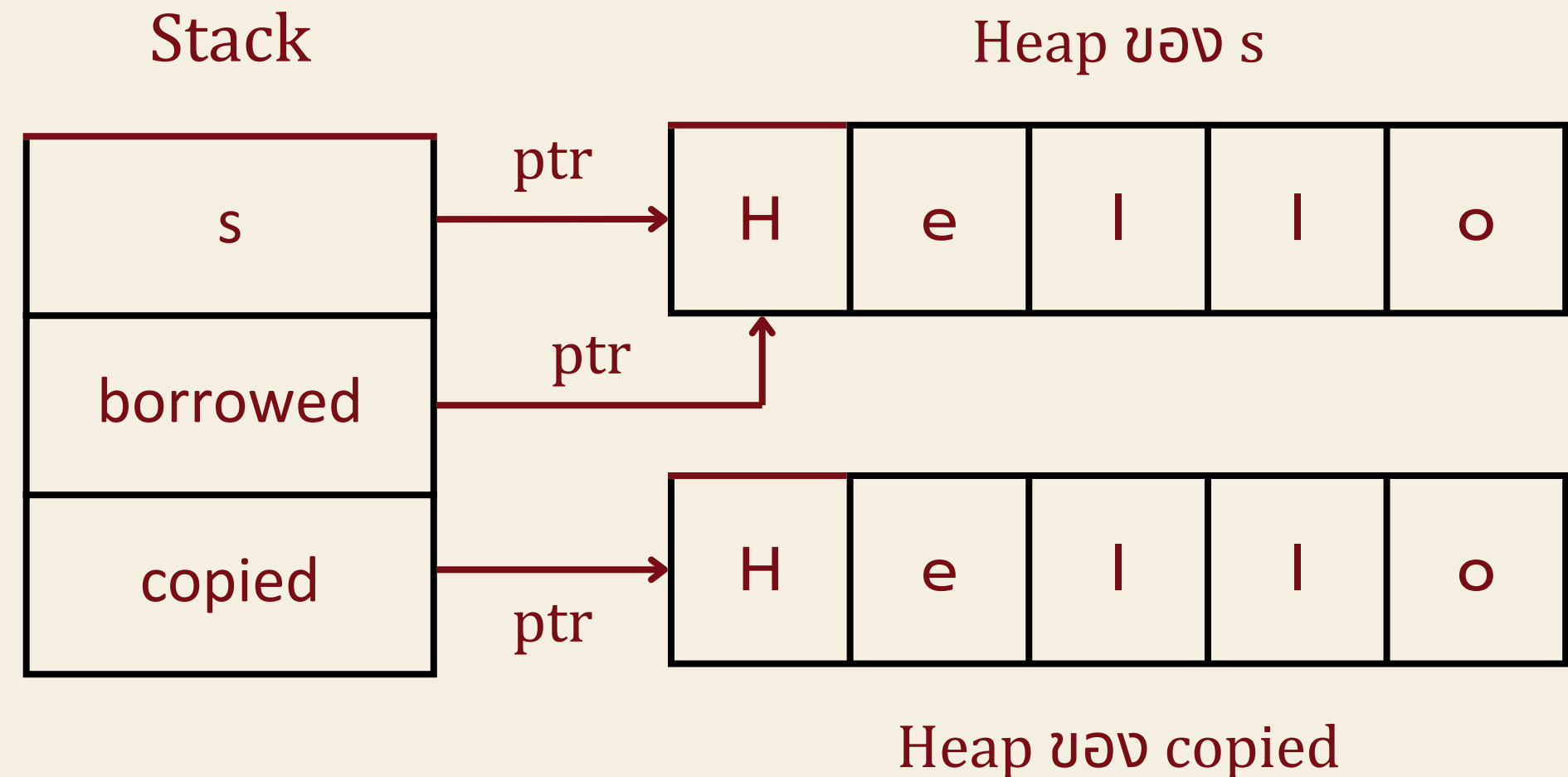
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```

Hello //ที่อยู่จาก Heap ของ s
Hello //ที่อยู่จาก Heap ของ s
Hello //ที่อยู่จาก Heap ของ copied

```



SLICES (CONCEPT)

- Slice ไม่ใช้การแย่ง owner หรือการแก้ค่าแต่อย่างใด แต่มันแค่การขออ่านค่าตรงตำแหน่งที่ i จนถึงตำแหน่งที่ j
- เช่น `&s[1..3]` แปลว่าจะขออ่านค่า ตั้งแต่ตัวที่ 1 จนถึงตัวที่ 2 ซึ่งมีความยาว = 2

```
1 fn main() {
2     let arr = [10, 20, 30, 40, 50];
3     let v = vec![10, 20, 30, 40, 50];
4
5     let s1: &[i32] = &arr[1..4]; //
6     let s2: &[i32] = &v[1..4];   //
7     let s3: &[i32] = &v;         //
8 }
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
[20 , 30 , 40] //ยืมอ่านค่าใน stack ของ array
[20 , 30 , 40] //ยืมอ่านค่าใน heap ของ vector
[10 , 20 , 30 , 40 , 50] //ก็อปปีมาเลยจากตัวตั้งต้น
```

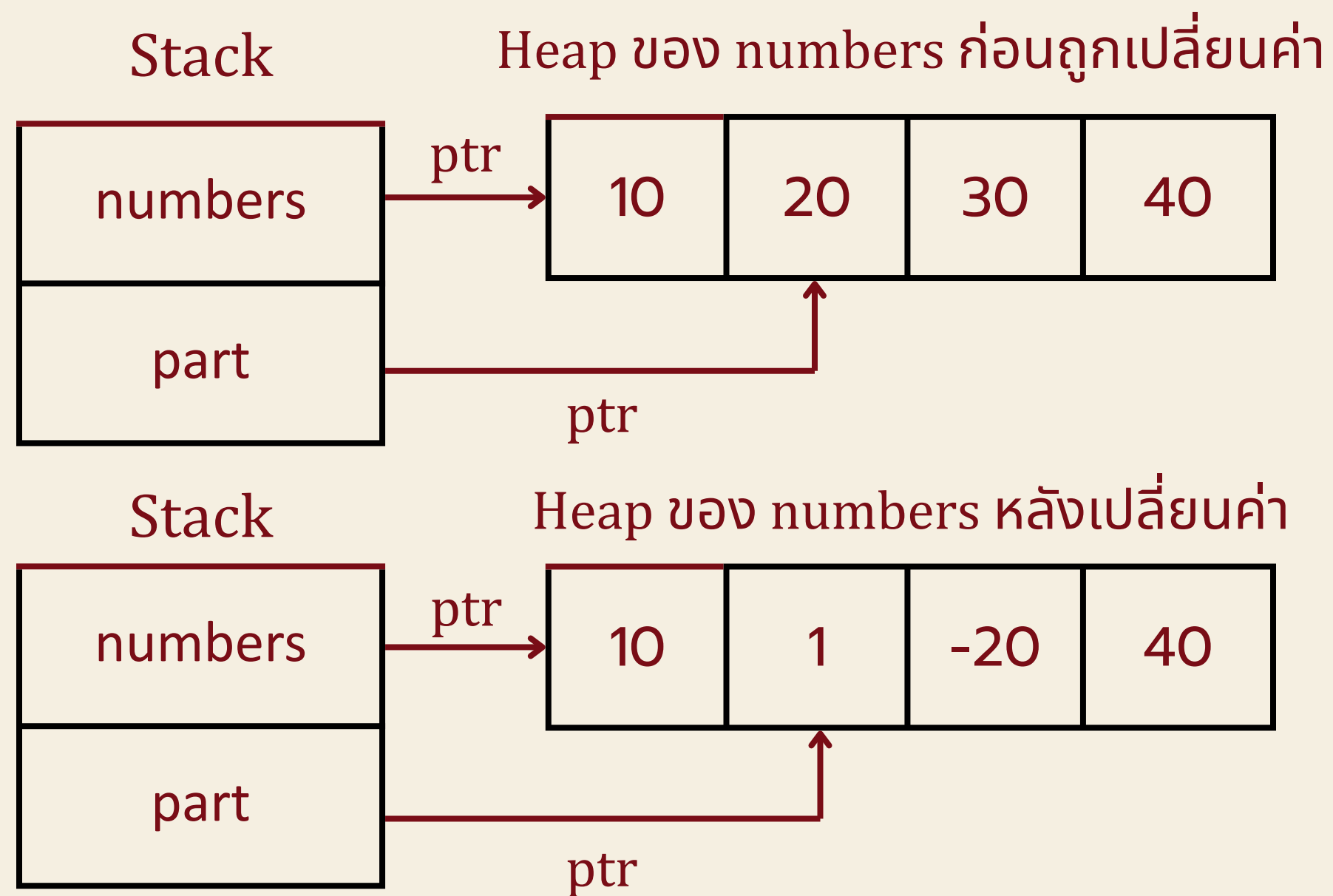
```
1 fn main() {
2     let s = String::from("Silpakorn");
3
4     let same = &s[..];
5     println!("Slices ได้ตัวเอง จะได้ {same}");
6
7     let silp = &s[..3];
8     println!("Slices ถึงตัวที่ 3 จะได้ {silp}");
9
10    let pako = &s[3..7];
11    println!("Slices ตั้งแต่ตัวที่ 3 - 7 จะได้ {pako}");
12
13    let rn = &s[7..];
14    println!("Slices ตั้งแต่ตัวที่ 7 เป็นต้นไป จะได้ {rn}");
15 }
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
Slices ได้ตัวเอง จะได้ Silpakorn
Slices ถึงตัวที่ 3 จะได้ Sil
Slices ตั้งแต่ตัวที่ 3 - 7 จะได้ pako
Slices ตั้งแต่ตัวที่ 7 เป็นต้นไป จะได้ rn
```

SLICES (CONCEPT)

- Mutable Slice : สามารถแก้ไขค่าที่ slice ออกมาจากข้อมูลหลักได้ หากแก้ไข ตัวข้อมูลหลักจะถูกแก้ไขตามไปด้วย
- ไม่สามารถ เพิ่ม - ลด ความยาวของตัว slice ได้



```

1 fn main() {
2     // Vec: ยืมเฉพาะช่วง index 1..3 แล้วแก้ค่าใน slice
3     let mut numbers = vec![10, 20, 30, 40];
4     let part: &mut [i32] = &mut numbers[1..3];
5     part[0] = 1;
6     part[1] = -20; // จะ panic เพราะ index 2 ไม่อยู่ใน slice
7     println!("Vec: {numbers:?}"); // [10, 1, -20, 40]
8
9     // String: ยืมช่วงที่เป็นขอบเขต UTF-8 แล้วแก้ตัวอักษร ASCII
10    // mutable str เปลี่ยนจำนวนไบต์หรือเพิ่ม/ลบตัวอักษรไม่ได้
11    let mut text = String::from("hello");
12    let part: &mut str = &mut text[1..4];
13    part.make_ascii_uppercase();
14    println!("String: {text}"); // hELLO
15 }

```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```

Vec: [10, 1, -20, 40]
String: hELLO

```

SLICE IS “FAT POINTER”

```

1  pub fn show_fat_pointer<T: std::fmt::Debug>(data: &[T]) {
2      let ptr = data.as_ptr();
3      let len = data.len();
4
5      println!("ptr : {:p} | len : {}", ptr, len);
6      println!("size of &[T] : {} bytes", std::mem::size_of::<[T]>());
7      println!("size of T : {} bytes", std::mem::size_of::<T>());
8
9      let s1 = &data[..data.len() - 2];
10     println!("slice ptr : {:p} | len : {}", s1.as_ptr(), s1.len());
11     println!("{:?}", s1);
12 }
13
14 pub fn slice_is_fat_pointer() {
15     let arr = [1, 2, 3, 4, 5, 6];
16     let vec = vec![1, 2, 3, 4, 5, 6];
17
18     println!("--- array ---");
19     println!("{:?}", arr);
20     show_fat_pointer(&arr);
21
22     println!("--- vector ---");
23     println!("{:?}", vec);
24     show_fat_pointer(&vec);

```

`&[T]` (slice) ไม่ได้เก็บแค่ที่อยู่ในหน่วยความจำ แต่เก็บ 2 อย่างคู่กัน:

- `ptr` คือที่อยู่ของสมาชิกตัวแรก
- `len` คือจำนวนสมาชิก

เรียกว่า **fat pointer** เพราะใหญ่กว่า pointer ธรรมดาที่เก็บแค่ที่อยู่อย่างเดียว (thin pointer)

`data.as_ptr()` และ `data.len()` ถึง `ptr` กับ `len` ออกมาจาก slice

`size_of::<&[T]>()` จะได้ 16 bytes บนเครื่อง 64-bit (pointer 8 + len 8) ซึ่งเป็น fat pointer

`size_of::<T>()` คือขนาดสมาชิก 1 ตัว (ในที่นี้ `i32` = 4 bytes)

SLICE IS “FAT POINTER”

```

1  pub fn show_fat_pointer<T: std::fmt::Debug>(data: &[T]) {
2      let ptr = data.as_ptr();
3      let len = data.len();
4
5      println!("ptr : {:p} | len : {}", ptr, len);
6      println!("size of &[T] : {} bytes", std::mem::size_of::<[T]>());
7      println!("size of T : {} bytes", std::mem::size_of::<T>());
8
9      let s1 = &data[..data.len() - 2];
10     println!("slice ptr : {:p} | len : {}", s1.as_ptr(), s1.len());
11     println!("{:?}", s1);
12 }
13
14 pub fn slice_is_fat_pointer() {
15     let arr = [1, 2, 3, 4, 5, 6];
16     let vec = vec![1, 2, 3, 4, 5, 6];
17
18     println!("--- array ---");
19     println!("{:?}", arr);
20     show_fat_pointer(&arr);
21
22     println!("--- vector ---");
23     println!("{:?}", vec);
24     show_fat_pointer(&vec);

```

ส่วน unsafe ใช้ `from_raw_parts(ptr, len - 2)` สร้าง slice ใหม่จาก ptr เดิม แต่ตัดความยาวลง 2 ผลคือได้ `[1, 2, 3, 4]` โดยไม่ได้ copy ข้อมูลเลย แค่เปลี่ยนตัวเลข len นี่คือหัวใจของ slice: เป็นแค่ "หน้าต่างมองข้อมูล" ไม่ได้เป็นเจ้าของข้อมูล

ข้อควรระวัง

- `fatptr.len - 2` ถ้า slice ยาวน้อยกว่า 2 จะเกิด underflow (panic ใน debug mode) ถ้าจะใช้จริงควรเช็คความยาวก่อน

STRING AND &STR

```

1  pub fn first_word(s: &str) -> &str { //(s: &String) -> usize
2      let bytes = s.as_bytes();
3      for (i, &item) in bytes.iter().enumerate() {
4          if item == b' ' {
5              return &s[0..i];}} // return i;
6      &s[..]}
7
8  pub fn string_and_str() {
9      let mut string = String::from("Silpakorn University");
10     let literal: &'static str = "Silpakorn";
11
12     let word = first_word(&string);
13     println!("word from String : {word}");
14     let word = first_word(literal);
15     println!("word from literal : {word}");
16     string.clear();
17     println!("string after clear : {:?}", string);
18
19     let th = String::from("รองเท้านารีคุณ");
20     println!("----- {} -----", th);
21     println!("len : {} bytes", th.len());
22     println!("char count : {} chars", th.chars().count());

```

(&String -> usize) คืบแค่ตำแหน่ง index ของช่องว่างแรก ปัญหาคือ index นี้ไม่ผูกกับสตริงเลย ถ้าเรียก string.clear() ที่หลัง index ก็ยังอยู่แต่ไม่มีความหมายแล้ว

(&str -> &str) คืบ slice ที่ชี้เข้าไปในสตริงเดิม ทำให้ borrow checker รู้ว่าผลลัพธ์ยังยึดสตริงอยู่ ถ้าสตริงต้นทางถูกแก้ไขหรือล้างขณะที่ slice ยังถูกใช้ จะ compile ไม่ผ่าน

ภาษาอังกฤษ 1 ตัว = 1 byte จึงเท่ากันหมด และตัด [0..1] ได้ตัวแรกพอดี

STRING AND &STR

```

17     println!("string after clear : {:?}", string);
18
19     let th = String::from("รองเท้านารีคุณ");
20         println!("----- {} -----", th);
21     println!("len : {} bytes", th.len());
22     println!("char count : {} chars", th.chars().count());
23     //println!("first letter : {}", &th[0..1]); // error
24
25     println!("----- is char boundary -----");
26     println!("1 byte : {}", th.is_char_boundary(1));
27     println!("3 bytes : {}", th.is_char_boundary(3));
28     println!("first letter : {}", &th[0..3]);
29
30     println!("----- fat pointer ownership -----");
31     let mut s1 = String::from("ญี่ปุ่น");
32     let s2 = "มาแล้ว";
33     s1.push_str(s2);
34     println!("{}", s1);
35     println!("s2 is {s2}");
36 }

```

ภาษาไทยตัวแรกกินที่ 3 bytes การใช้
 &th[0..1] จะ error เพราะ**ตัดกลางตัว
 byte** ที่ไม่สมบูรณ์

is_char_boundary ใช้ถามว่า "ตำแหน่ง
 byte นี้เป็นจุดเริ่มของตัวอักษรไหม"

s1 เป็น String คือ เป็นเจ้าของ ข้อมูล
 แก้ไข/ขยายได้

s2 เป็น &str คือ fat pointer (ptr +
 len) แค่ "ยืมดู" ข้อมูล เป็นเจ้าของไม่ได้

push_str แะยืม s2 มา copy ข้อความเข้า
 heap ของ s1 จึงไม่ได้เอา s2 ไป และยังใช้
 s2 ต่อได้

MUTABLE SLICES

```

1  pub fn transform_even_odd(slice: &mut [i32]) {
2      for x in slice.iter_mut() {
3          if *x % 2 == 0 {
4              *x *= 2;
5          } else {
6              *x -= 1;
7          }}}
8
9  pub fn mutable_slices() {
10     let mut numbers = [1, 2, 3, 4, 5, 6];
11     println!("before: {:?}", numbers);
12     transform_even_odd(&mut numbers);
13     println!("after: {:?}", numbers);
14
15     //let s1 = &mut numbers[..2];
16     //let s2 = &mut numbers[1..];    // error
17     //transform_even_odd(s1);
18     //transform_even_odd(s2);
19

```

`iter_mut()` ให้ `&mut i32` ของแต่ละตัว

`*x` คือการ "ตามไปที่ค่าจริง" เพื่ออ่าน/แก้

ฟังก์ชันรับ `&mut [i32]` จึงแก้ไขข้อมูลของคนเรียกได้โดยตรง ไม่ต้อง `return`

ต้องประกาศ `let mut` ไม่ขึ้นแก้ไขไม่ได้

`&mut numbers` (ชนิด `&mut [i32; 6]`) ถูกแปลงเป็น `&mut [i32]` ให้อัตโนมัติ

MUTABLE SLICES

```

9  pub fn mutable_slices() {
10     let mut numbers = [1, 2, 3, 4, 5, 6];
11     println!("before: {:?}", numbers);
12     transform_even_odd(&mut numbers);
13     println!("after: {:?}", numbers);
14
15     //let s1 = &mut numbers[..2];
16     //let s2 = &mut numbers[1..];    // error
17     //transform_even_odd(s1);
18     //transform_even_odd(s2);
19
20     let (left, right) = numbers.split_at_mut(3);
21     // println!("numbers : {:?}", numbers); // error
22     left[0] = 111;
23     right[0] = 999;
24     println!("left : {:?}", left);
25     println!("right : {:?}", right);
26     // println!("numbers : {:?}", numbers);
27 }

```

s1 กับ s2 ถูกใช้ต่อหลังจากนั้นทั้งคู่ Rust จึงไม่ยอมให้คอมไพล์

หมายเหตุ: จริงๆ Rust ไม่ได้ดูว่า range กับกันจริงหรือเปล่า แค่เห็นว่า numbers[..] ถูก &mut ยืมซ้ำก็ error แล้ว แม้ range จะไม่ทับกัน (เช่น[..2] กับ [3..]) ก็ยัง error เพราะมัน**ตรวจ range ตอนคอมไพล์ไม่ได้**

ใช้ split_at_mut แบ่ง slice เป็น 2 ส่วนที่ ไม่ทับกันแน่นอน

ตราบใดที่ left/right ยังถูกใช้อยู่ข้างล่าง numbers ถือว่า ถูกยืมแบบ mutable อยู่ จะอ่าน numbers ตรงๆ ไม่ได้

SLICE METHODS (ZERO-COPY VIEWS)

```

1  pub fn slice_methods() {
2      let nums = [1, 2, 3, 4, 5, 6, 7];
3
4      match &nums[..] {
5          [first, .., last] => println!("first / last : {first} / {last}"),
6          [only] => println!("only one: {only}"),
7          [] => println!("empty"),
8      }
9      // println!("first / last : {:?} / {:?}", nums.first(), nums.last());
10     match &nums[..] {
11         [first, rest @ ..] => println!("first / rest : {first} / {rest:?}"),
12         [] => {}
13     }
14     println!(".get(10) : {:?}", nums.get(10));
15     println!(".split_at(3) : {:?}", nums.split_at(3));
16     println!(".chunks(3) : {:?}", nums.chunks(3).collect::<Vec<_>>());
17     println!(".windows(3) : {:?}", nums.windows(3).collect::<Vec<_>>());
18     println!(".contains(&4) : {}", nums.contains(&4));
19
20     let owned: Vec<i32> = nums[..3].to_vec();
21     println!("to_vec : {:?}", owned);
22 }

```

first / last/ rest หาข้อมูลตัวแรก/สุดท้ายในอาร์เรย์/ตัวที่เหลือ (ถ้า slice ว่างจะได้ None)

get(x) index เกินขอบเขตได้ None ไม่ panic (ต่างจาก nums[10] ที่ panic)

split_at(x) แบ่งเป็น 2 slice ที่ index x คืนเป็น tuple (ไม่ copy)

chunks(x) ตัดเป็นก้อนละ x ไม่ซ้อนกัน ก้อนสุดท้ายเหลือเท่าไรก็เท่านั้น

windows(x) "หน้าต่าง" ขนาด x เลื่อนทีละ 1 ซ้อนกัน

contains(&x) เช็คว่ามีค่านี้อยู่ไหม รับ reference และวนหาแบบเรียงทีละตัว

SLICE METHODS (ZERO-COPY VIEWS)

```

1  pub fn slice_methods() {
2      let nums = [1, 2, 3, 4, 5, 6, 7];
3
4      match &nums[..] {
5          [first, .., last] => println!("first / last : {first} / {last}"),
6          [only] => println!("only one: {only}"),
7          [] => println!("empty"),
8      }
9      // println!("first / last : {:?} / {:?}", nums.first(), nums.last());
10     match &nums[..] {
11         [first, rest @ ..] => println!("first / rest : {first} / {rest:?}"),
12         [] => {}
13     }
14     println!(".get(10) : {:?}", nums.get(10));
15     println!(".split_at(3) : {:?}", nums.split_at(3));
16     println!(".chunks(3) : {:?}", nums.chunks(3).collect::<Vec<_>>());
17     println!(".windows(3) : {:?}", nums.windows(3).collect::<Vec<_>>());
18     println!(".contains(&4) : {}", nums.contains(&4));
19
20     let owned: Vec<i32> = nums[..3].to_vec();
21     println!("to_vec : {:?}", owned);
22 }

```

`to_vec()` คัดลอก ไปสร้าง Vec ใหม่บน heap ที่เป็นเจ้าของข้อมูลเอง

เมธอดส่วนใหญ่ (`first`, `get`, `split_at`, `chunks`, `windows`) คั้น slice/reference = ยืมข้อมูลเดิม ไม่ **copy**

มีแค่ `to_vec()` ที่ **copy** จริง

PPL PERSPECTIVE — มุมมองเชิง PROGRAMMING LANGUAGE

Syntax

Rust ใช้ slice เพื่ออ้างอิงข้อมูลบางส่วนของ collection โดยไม่ต้องสร้าง collection ใหม่ขึ้นมา รูปแบบที่พบบ่อยคือ `&collection[start..end]`

ตัวอย่าง

```
let numbers = [10, 20, 30, 40, 50];  
let part = &numbers[1..4];
```

ดังนั้น Syntax ของ Slice จะเน้นการใช้ & ร่วมกับช่วง `[start..end]` เพื่อสร้าง reference ไปยังข้อมูลเดิม

PPL PERSPECTIVE — มุมมองเชิง PROGRAMMING LANGUAGE

Semantics

Slice มีความหมายว่าเป็น ส่วนหนึ่งของข้อมูลเดิม ไม่ใช่ข้อมูลชุดใหม่

ตัวอย่าง

```
let data = [10, 20, 30, 40, 50];  
let part = &numbers[1..4];
```

part หมายถึงข้อมูล 20, 30, 40 แต่ข้อมูลยังอยู่ใน data เหมือนเดิม
แนวคิดสำคัญคือ Slice เป็น borrowed reference จึงไม่ได้เป็นเจ้าของข้อมูล
ดังนั้น Semantics ของ Slice คือ การอ้างอิงข้อมูลเดิมบางช่วงผ่าน borrowing โดยไม่
โอน ownership

PPL PERSPECTIVE — มุมมองเชิง PROGRAMMING LANGUAGE

Type System

Rust มี Type System ที่ทำงานร่วมกับ Slice และ Ownership อย่างชัดเจน

ตัวอย่างชนิดที่เกี่ยวข้อง

```
let arr: [i32; 5] = [1, 2, 3, 4, 5];  
let slice: &[i32] = &arr[1..4];  
let text: &str = "Hello Rust";  
let vec: Vec<i32> = vec![1, 2, 3, 4, 5];  
let vec_slice: &[i32] = &vec[1..4];
```

ดังนั้น Rust จึงใช้ Type System เพื่อแยกให้ชัดว่าอะไรเป็นเจ้าของข้อมูล และอะไรเป็นเพียง reference ที่ยืมข้อมูลมาใช้

PPL PERSPECTIVE — มุมมองเชิง PROGRAMMING LANGUAGE

Memory / Resource

Rust มีการจัดการ Memory โดยใช้แนวคิด Ownership, Borrowing และ Lifetime แทนการใช้ Garbage Collector

ตัวอย่าง

```
let numbers = vec![10, 20, 30, 40, 50];  
let part = &numbers[1..4];
```

ในตัวอย่างนี้ numbers เป็นเจ้าของ Vec และข้อมูลของ Vec โดยทั่วไปจะเก็บอยู่บน Heap ส่วนตัวแปร part เป็น slice reference ที่ชี้ไปยังข้อมูลบางส่วนของ numbers จุดสำคัญคือ Slice ไม่ได้เป็นเจ้าของข้อมูล และเมื่อ numbers หมด Scope ข้อมูลที่ part อ้างอิงอยู่ก็ไม่สามารถถูกใช้งานต่อได้ เพราะ Rust ใช้ Borrow Checker ป้องกันไม่ให้เกิดการอ้างอิงข้อมูลทั้งหมดอายุแล้ว

PPL PERSPECTIVE — มุมมองเชิง PROGRAMMING LANGUAGE

Abstraction

Slice ช่วยให้โปรแกรมเมอร์สนใจเฉพาะ “ข้อมูลช่วงที่ต้องการ” โดยไม่ต้องจัดการรายละเอียดการสร้าง collection ใหม่

ตัวอย่าง

```
fn print_data(data: &[i32]) {  
    println!("{:?}", data);  
}
```

ฟังก์ชันนี้สามารถรับได้ทั้ง
`let arr = [1, 2, 3, 4];`
`let vec = vec![5, 6, 7, 8];`

เพราะทั้ง Array และ Vector สามารถถูกยืมมาเป็น `&[i32]` ได้

จึงเป็นการสร้าง Abstraction ให้ฟังก์ชันทำงานกับ “ลำดับของข้อมูล” โดยไม่ต้องสนใจว่าเจ้าของข้อมูลจริง ๆ เป็น Array หรือ Vector

PPL PERSPECTIVE — มุมมองเชิง PROGRAMMING LANGUAGE

Scope

ตัวแปรและ reference จะใช้
งานได้ภายใน Scope ที่กำหนด
ตัวอย่าง

```
let data = vec![1, 2, 3];

{
    let part = &data[0..2];
}
```

part จะมี Scope อยู่ภายใน
block เท่านั้น

Binding

Rust ใช้ let สำหรับ Binding
ตัวแปรกับค่า
ตัวอย่าง

```
let data = [1, 2, 3, 4];
let part = &data[1..3];
```

ในที่นี้ part ถูก Binding ให้
เป็น reference ไปยังส่วน
หนึ่งของ data

Lifetime

Slice ต้องไม่สามารถมีอายุยาว
กว่า Owner ของข้อมูล
ตัวอย่าง

```
let data = vec![1, 2, 3];
let part = &data[0..2];
```

Lifetime ของ part จึงขึ้นอยู่กับ data
แนวคิดนี้ช่วยให้ Rust ตรวจ
สอบปัญหาเกี่ยวกับ Memory
ตั้งแต่ Compile Time

PPL PERSPECTIVE — มุมมองเชิง PROGRAMMING LANGUAGE

Why Rust?

Rust ออกแบบมาให้สามารถควบคุม Memory ได้ดี แต่ในขณะเดียวกันก็พยายามป้องกันปัญหาที่เกิดจากการจัดการ Memory สำหรับ Slice จุดเด่นคือสามารถ เข้าถึงข้อมูลบางส่วนโดยไม่ต้อง Copy ข้อมูลใหม่

ตัวอย่าง

```
let data = vec![10, 20, 30, 40, 50];  
let part = &data[1..4];
```

part เพียงแค่อ้างอิงข้อมูลเดิม ทำให้ลดการสร้างข้อมูลซ้ำและช่วยเรื่อง Performance

ดังนั้น Rust ให้ความสำคัญกับ Memory Safety โดยใช้ Ownership, Borrowing และ Lifetime ในการจัดการหน่วยความจำ แนวคิดเหล่านี้ช่วยป้องกันปัญหาที่พบบ่อยในภาษาแบบ Manual Memory Management เช่น dangling reference คือ reference ที่ชี้ไปยังข้อมูลที่หมดอายุแล้ว, use-after-free คือการใช้ข้อมูลหลังจาก Memory ถูกคืนไปแล้ว, double free คือการคืน Memory เดิมมากกว่าหนึ่งครั้ง และ invalid memory access คือการเข้าถึง Memory ในตำแหน่งที่ไม่ถูกต้อง เช่น การเข้าถึง Array เกินขอบเขต

RUST VS PYTHON

Aspect	Rust	Python
Syntax	ใช้ slice เช่น &collection[start..end]; &[T] เป็น shared slice และ &str เป็น string slice	ใช้ slicing เช่น data[1:4] และ text[1:4]
Semantics / Behavior	Slice เป็น borrowed reference ไปยังข้อมูลเดิม ไม่ได้สร้างข้อมูลใหม่ และไม่เป็นเจ้าของข้อมูล	Slice ของ list, str โดยทั่วไปได้ข้อมูลใหม่ ส่วนตัวแปรเดิมยังเป็นเจ้าของข้อมูลของตัวเอง
Type System	Static + Strong typing มีชนิดชัดเจน เช่น &[i32], &str, Vec<i32>	Dynamic typing ชนิดของตัวแปรถูกตรวจสอบตอน Runtime
Memory Management	ใช้ Ownership, Borrowing และ Lifetime โดยไม่มี Garbage Collector	จัดการ Memory อัตโนมัติตาม implementation เช่น reference counting/garbage collection
Safety	เน้น Memory Safety ตั้งแต่ Compile Time เช่น ป้องกัน dangling reference และ use-after-free	มีการจัดการ Memory อัตโนมัติและตรวจสอบหลายอย่างตอน Runtime

ANALYSIS

ความแตกต่างที่สำคัญระหว่าง Rust กับภาษาอื่นอยู่ที่ แนวคิดในการจัดการ Memory และการออกแบบ Slice โดย Rust พยายามให้โปรแกรมสามารถเข้าถึงข้อมูลเดิมได้โดยไม่ต้อง Copy ข้อมูล แต่ยังคงตรวจสอบความปลอดภัยของการอ้างอิงตั้งแต่ Compile Time

Rust vs. Python

Python ใช้ Syntax ของ Slice ที่ง่าย เช่น `data[1:4]` และจัดการ Memory ให้อัตโนมัติ ทำให้เขียนโปรแกรมได้สะดวก แต่ไม่ได้บังคับให้โปรแกรมเมอร์ระบุ Ownership หรือความสัมพันธ์ของ Reference แบบ Rust การออกแบบของ Rust จึงเน้นให้ผู้เขียนโปรแกรมควบคุมการยืมข้อมูลได้ชัดเจนขึ้น เพื่อให้เกิด Memory Safety โดยไม่ต้องพึ่ง Garbage Collector

ANALYSIS

เหตุผลด้านการออกแบบภาษา

ใน Rust Slice คือการอ้างอิงไปยังข้อมูลบางส่วนของข้อมูลเดิม โดยไม่จำเป็นต้องสร้างหรือ Copy ข้อมูลชุดใหม่ขึ้นมา ทำให้ประหยัด Memory และช่วยให้การทำงานมีประสิทธิภาพมากขึ้น อย่างไรก็ตาม Slice ไม่ได้เป็นเจ้าของข้อมูลนั่นเอง แต่เป็นเพียงการ Borrow ข้อมูลจากตัวแปรที่เป็น Owner

Rust จึงใช้แนวคิด Ownership, Borrowing และ Lifetime เพื่อควบคุมว่าข้อมูลสามารถถูกอ้างอิงและใช้งานได้นานแค่ไหน โดย Compiler จะตรวจสอบกฎเหล่านี้ตั้งแต่ Compile Time เพื่อป้องกันปัญหาที่เกี่ยวข้องกับ Memory เช่น การอ้างอิงข้อมูลที่หมดอายุหรือการใช้ข้อมูลผิดช่วงเวลา

ดังนั้น จุดเด่นของ Rust ไม่ได้อยู่ที่การมี Slice เพียงอย่างเดียว แต่คือการออกแบบให้ การอ้างอิงข้อมูลที่มีประสิทธิภาพสามารถใช้งานร่วมกับระบบ Ownership ได้อย่างปลอดภัย ซึ่งเป็นแนวคิดที่แตกต่างจาก Python และ Java ที่เน้นการจัดการ Memory อัตโนมัติ และแตกต่างจาก C/C++ ที่ให้อิสระในการจัดการ Memory มากกว่า

COMMON MISTAKES — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
fn main() {  
    let s1 = String::from("hello"); // heap-allocated  
    let s2 = s1; // move, s1 ใช้ไม่ได้อีก  
    println!("{}", s1); // compile error! เพราะเป็นการคัดลอกเพียง  
                        // 24 bytes บน stack ไปที่ s2 ไม่ได้คัดลอก heap ไปด้วย  
}
```

✓ Correct

```
fn main() {  
    let s1 = String::from("hello"); // heap-allocated  
  
    // คัดลอก heap ด้วย (deep copy) แพงกว่า แต่ s1 ยังใช้ได้  
    let s2 = s1.clone();  
  
    // ยืม (borrow) ไม่ย้ายความเป็นเจ้าของ  
    let s2 = &s1;  
  
    // หรือใช้ s2 อย่างเดียว แล้วเลิกใช้ s1  
}
```

ทำไมถึงผิด? (Why?)

ผิดเพราะ `let s2 = s1` เป็นการ ย้าย ownership (Move) จาก `s1` ไปให้ `s2` ทำให้ `s1` หหมดสิทธิ์ใช้งาน จึงไม่สามารถ `println!("{}", s1)` ได้อีก

COMMON MISTAKES — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
fn main() {  
    let numbers = [10, 20, 30];  
  
    let x = &numbers[1..4];  
}
```

✓ Correct

```
fn main() {  
    let numbers = [10, 20, 30];  
  
    let x = &numbers[0..3];  
}
```

ทำไมถึงผิด? (Why?)

กำหนดช่วง Slice เกินขนาดของข้อมูล

COMMON MISTAKES — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
fn main() {  
    let numbers = [10, 20, 30];  
  
    let x = &numbers[0..2];  
  
    x[0] = 100;  
}
```

✓ Correct

```
fn main() {  
    let mut numbers = [10, 20, 30];  
  
    let x = &mut numbers[0..2];  
  
    x[0] = 100;  
}
```

ทำไมถึงผิด? (Why?)

ใน Rust ถ้าเราใช้ & เพื่อยืมข้อมูล จะเป็นการยืมแบบอ่านอย่างเดียว ดังนั้นเราจะไม่สามารถเปลี่ยนค่าข้างใน Slice ได้

COMMON MISTAKES — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
fn main() {  
    let mut numbers = vec![1, 2, 3];  
  
    for num in &numbers {  
        if num == 2 {  
            numbers.push(4);  
        }  
    }  
}
```

✓ Correct

```
fn main() {  
    let mut numbers = vec![1, 2, 3];  
  
    let len = numbers.len();  
  
    for i in 0..len {  
        if numbers[i] == 2 {  
            numbers.push(4);  
        }  
    }  
  
    println!("{:?}", numbers);  
}
```

ทำไมถึงผิด? (Why?)

พยายามแก้ไขหรือเพิ่มข้อมูลลงใน Vector ในขณะที่ยังมีตัวแปรชี้มาอ่านข้อมูลใน Vector นั้นอยู่

CHALLENGE — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

กำหนดให้ `numbers` มีค่า `[10, 20, 30, 40, 50]` จงใช้ `Slice` เพื่อเลือกข้อมูล 20, 30, 40 และเปลี่ยนค่าเป็น 200, 300, 400 จากนั้นแสดงผลข้อมูลที่ถูกละเปลี่ยนแปลงแล้ว โดยห้ามเปลี่ยนค่าใน `numbers` โดยตรง ให้แก้ไขข้อมูลผ่าน `Slice` เท่านั้น



Hint

ให้ใช้ `Slice` เพื่อเลือกเฉพาะข้อมูล 20, 30, 40 จาก `numbers` โดยกำหนดช่วงให้ถูกต้อง จากนั้นใช้การยืมแบบ `Mutable` เพื่อให้สามารถแก้ไขค่าผ่าน `Slice` ได้ และแสดงผล

CHALLENGE — คำถามท้าทายผู้ฟัง

Solution

```
fn main() {  
    let mut numbers = [10, 20, 30, 40, 50];  
  
    let x = &mut numbers[1..4];  
  
    x[0] = 200;  
    x[1] = 300;  
    x[2] = 400;  
  
    println!("{:?}", x);  
}
```

CHALLENGE — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

จงสร้าง Vector เก็บชุดตัวเลขจำนวนเต็ม ขนาด N ตัว คือ [1,4,6,9] จากนั้นให้ทำการตรวจสอบข้อมูลตัวเลขเดิมใน Vector: ถ้าเจอ เลขคู่ ให้ทำการเพิ่ม (Push) ค่า $x * 2$ ต่อท้ายเข้าไปใน Vector ถ้าเจอ เลขคี่ ให้ทำการเพิ่ม (Push) ค่า $x / 2$ ต่อท้ายเข้าไปใน Vector
เงื่อนไข: ต้องตรวจสอบเช็คครบเฉพาะ N ตัวแรกเท่านั้น

จากโจทย์ถ้าต้องการให้ Compile ผ่าน ตัวเลือกข้อไหนถูกต้อง

 **Hint** ห้ามยืมอ่านข้อมูลค้างไว้ทั้ง Loop แล้วสั่งเพิ่มขนาด Vector พร้อมกัน

- A** `for val in &numbers[..len] { if val % 2 == 0 { numbers.push(val * 2); } else { numbers.push(val / 2); } }`

B `let slice_vals = numbers[..len].to_vec(); for val in slice_vals { if val % 2 == 0 { numbers.push(val * 2); } else { numbers.push(val / 2); } }`

C `for i in 0..numbers.as_slice().len() { let val = &mut numbers[i]; if *val % 2 == 0 { numbers.push(*val * 2); } else { numbers.push(*val / 2); } }`

D `for i in 0..numbers.len() { let val = numbers[i]; if val % 2 == 0 { numbers.push(val * 2); } else { numbers.push(val / 2); } }`

CHALLENGE — คำถามท้าทายผู้ฟัง

Solution

```
let slice_vals = numbers[..len].to_vec();  
for val in slice_vals {  
    if val % 2 == 0 {  
        numbers.push(val * 2);  
    } else {  
        numbers.push(val / 2);  
    }  
}
```

B

โค้ดนี้คัดลอกข้อมูล N ตัวแรกออกจาก numbers ไปสร้างเป็น Vector ใหม่ slice_vals เพื่อไม่ให้ติดกฎ Borrow Checker ของ Rust ทำให้สามารถวนลูปเช็คเลขคู่/คี่ แล้วสั่ง push ผลลัพธ์ต่อท้ายเข้า numbers ตัวเดิมได้อย่างปลอดภัยและตรงตามเงื่อนไข N ตัวแรกครบถ้วนครับ

KEY TAKEAWAYS — สรุปสิ่งที่ควรจำ

- 1 Memory Model : มี 2 ส่วน คือ Stack ไว้เก็บค่าตัวแปรขนาดคงที่ระหว่างการ compile
Heap เก็บค่าตัวแปรที่ขนาดยืดหยุ่นระหว่างการ compile
- 2 การ Slice & แค่เป็นการยืมอ่านค่าใน memory ของตัวที่สนใจ ต่างจาก .clone() ที่จะสร้าง heap ใหม่เป็นของตัวเอง
- 3 หากใช้ mutable slice แล้วแก้ไขในตัว slice จะส่งผลต่อค่าหลักด้วย เนื่องจากเป็นการแก้ไขก่อนข้อมูลที่เป็นตัวเดียวกัน
- 4 Rust ทำ Slice โดยใช้ Ownership/Borrowing ควบคุมการใช้ Memory และตรวจสอบความปลอดภัยตั้งแต่ compile time

References — เอกสารอ้างอิง

- The Rust Programming Language — Understanding Ownership, References and Borrowing, Slices <https://doc.rust-lang.org/book/>
- The Rust Reference — Slice Types & Reference Types <https://doc.rust-lang.org/reference/types/slice.html>
- Crate std - Primitive Type slice <https://doc.rust-lang.org/std/primitive.slice.html>
- Python Documentation — Common Sequence Operations / Slicing <https://docs.python.org/3/library/stdtypes.html>
- cppreference — C / C++ Arrays, Pointers and std::span <https://en.cppreference.com/w/c/language/array>
<https://en.cppreference.com/w/cpp/container/span>
- Oracle Java Documentation — Arrays <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html>
- Kodekloud - The Slice Type <https://notes.kodekloud.com/docs/Rust-Programming/Ownership/The-Slice-Type/page>
- Kodekloud - Rules for Slices <https://notes.kodekloud.com/docs/Rust-Programming/Ownership/Rules-for-Slices/page>
- The Rust Programming Language - Storing UTF-8 Encoded Text with Strings <https://doc.rust-lang.org/book/ch08-02-strings.html>

GitHub Repository: github.com/670710296/rust-tutorial-2569/

Questions & Answers

ក្រុម [13] — [Slices & Memory Model]